

Build Record — Prompts Transcript

This is the "build record" submission item: the sequence of prompts given to the AI coding agent (Claude Code) across the backend build, the frontend build, and the post-build validation/refinement session. Prompts are reproduced verbatim (typos included) in the order they were given. Short -> notes after each phase summarize the outcome factually, without expanding on the actual conversation.

Part 1 — Backend

The backend was originally built across an earlier agent session using the 15 sequential prompts below (from `Gold_Trading_Backend_Coding_Agent_Prompts.pdf`). That session ran out of budget partway through. This session was handed the same PDF and the existing codebase, and asked to verify, complete, and fix it.

Kickoff prompt for this session

```
C:\Users\Haroon\source\repos\AsasaGoldTrading this path of my working directory i worked on this project but my codinng agents limit got exhaust so i give you all verify them and implement missing one and also resolve errors for now leave deployment part file:///C:/Users/Haroon/Downloads/Gold_Trading_Backend_Coding_Agent_Prompts.pdf
```

-> *Result: audited the existing Clean Architecture backend against all 15*

prompts below, deleted a superseded duplicate flat-structure project, fixed dead code and a duplicated constant, implemented the missing Prompt 12 (demo controls) and Prompt 13 (full test suite — went from 22 to 70 tests), added a missing GET `/api/pricing/current` endpoint, fixed enum serialization, and verified the whole flow live in a browser.

The 15 prompts from `Gold_Trading_Backend_Coding_Agent_Prompts.pdf`

Prompt 1 — Architecture and Planning

```
I am building a Founding Engineer assessment project: a single-user demo for buying and selling gold.
```

```
IMPORTANT: This project must NOT use a database.  
Use an in-memory architecture instead.
```

My stack:

Backend:

- ASP.NET Core Web API
- C#
- .NET 8
- Clean Architecture, but lightweight
- In-memory repositories/state

Requirements:

1. Seed customer PKR wallet, customer gold holdings, and platform gold inventory.
2. Show live 24K gold price in PKR per gram.
3. Primary pricing provider: PakGold. Fallback: GoldPrice.org.
4. Server-side price caching with configurable cache interval.
5. Disable trading if neither source is trusted.
6. Pricing formulas must be configurable:
Buy = $\max(\text{MarketPrice} \times \text{BuyMarkupMultiplier}, \text{BuyPriceGuardrail})$
Sell = $\text{MarketPrice} \times \text{SellMarkdownMultiplier}$
7. User can enter PKR or gold grams.
8. Server owns the locked quote.
9. Quote duration is configurable and defaults to 75 seconds.
10. Same quote confirmed twice must create only one trade.
11. Handle insufficient cash, gold, inventory, expired quote, repeated confirmation and unavailable pricing.
12. Use centralized constants for user-facing messages.
13. No magic numbers or strings.
14. No database, Redis, Kafka, or unnecessary microservices.

Before writing code provide:

- Architecture diagram
- Folder structure
- Domain model
- In-memory state design
- Repository interfaces
- Thread-safety strategy
- API endpoints
- Configuration design
- Quote lifecycle
- Idempotency strategy

Do NOT write implementation yet. Wait for approval.

Prompt 2 — Backend Foundation

Now implement the backend foundation.

Create:

```
src/  
GoldTrading.Api  
GoldTrading.Application  
GoldTrading.Domain  
GoldTrading.Infrastructure
```

```
tests/  
GoldTrading.Application.Tests  
GoldTrading.IntegrationTests
```

IMPORTANT: There must be NO database and no EF Core.

Responsibilities:

Domain:

- Core entities
- Enums
- Business rules

Application:

- Use cases
- Interfaces
- DTOs
- Validation

Infrastructure:

- In-memory implementations
- External price providers
- Caching
- Synchronization

API:

- Controllers
- Middleware
- Dependency Injection
- Configuration

Create centralized:

```
Constants/  
ErrorMessages  
SuccessMessages  
ValidationMessages  
ApiMessages
```

```
Configuration/  
PricingOptions  
QuoteOptions  
TradingOptions  
SeedDataOptions
```

All business values must come from strongly typed configuration.

Do not hardcode quote duration, cache duration, markup, markdown, guardrail, or seed balances.

Add DI registrations and explain the dependency direction.

Prompt 3 — Domain Model

Now implement the domain model.

Create:
CustomerAccountState
MarketPrice
Quote
Trade
TradeReceipt

Enums:
TradeType: Buy, Sell
InputType: Pkr, Gold
QuoteStatus: Active, Confirmed, Expired
PricingSource: PakGold, GoldPriceOrg

Use decimal for PKR, gold grams and prices. Never use float or double.

CustomerAccountState contains:
- Customer PKR balance
- Customer gold balance
- Platform gold inventory

Quote contains:
- Id
- TradeType
- InputType
- LockedMarketPrice
- CustomerPricePerGram
- PkrAmount
- GoldAmountInGrams
- PricingSource
- PriceRetrievedAtUtc
- CreatedAtUtc
- ExpiresAtUtc
- Status

Trade contains:
- Id
- QuoteId
- TradeType
- PkrAmount
- GoldAmountInGrams
- PricePerGram
- CompletedAtUtc

Keep Domain independent from Infrastructure.
Add focused unit tests.

Prompt 4 — In-Memory Repositories and State

Now implement the in-memory persistence layer.

There is NO database.

Create interfaces:
IAccountStateRepository
IQuoteRepository
ITradeRepository

Implement:
InMemoryAccountStateRepository
InMemoryQuoteRepository
InMemoryTradeRepository

Requirements:
- One seeded customer account state
- Quotes stored in memory
- Trades stored in memory
- Trade retrievable by QuoteId
- Only one trade per QuoteId

Thread safety is important.

Create a dedicated synchronization strategy for trade settlement, such as:
ITradeSettlementLock / TradeSettlementLock using SemaphoreSlim.

Do not use one unnecessary global lock around the entire application.

Register stateful repositories as Singleton so state survives for the application process lifetime.

Seed values from SeedDataOptions, never hardcoded.

Explain:
1. Why Singleton is required
2. What happens after restart
3. Why this is acceptable for a single-instance demo
4. Production limitations.

Prompt 5 — Strongly Typed Configuration

Implement strongly typed configuration.

PricingOptions:
- CacheDurationMinutes
- BuyMarkupMultiplier
- SellMarkdownMultiplier
- BuyPriceGuardrail

QuoteOptions:
- DurationSeconds

SeedDataOptions:
- InitialCustomerPkrBalance
- InitialCustomerGoldGrams
- InitialPlatformGoldInventoryGrams

Bind all options from appsettings.json.

Validate options during startup.

Invalid examples:

- Cache duration ≤ 0
- Quote duration ≤ 0
- Invalid multipliers
- Negative balances
- Negative guardrail

Do not scatter IConfiguration calls through business services.
Use IOptions or IOptionsMonitor appropriately.

Prompt 6 — Live Pricing and Fallback

Implement the live pricing system.

Create:

IGoldPriceProvider

Return normalized:

MarketPriceResult:

- PricePerGram
- Currency
- Purity
- Source
- RetrievedAtUtc
- IsTrusted

Implement:

PakGoldPriceProvider

GoldPriceOrgPriceProvider

Create GoldPriceService.

Responsibilities:

1. Return cached trusted price while valid.
2. Fetch only after configured cache expiry.
3. Try PakGold first.
4. Fallback to GoldPrice.org.
5. Normalize to 24K PKR per gram.
6. Store selected price in in-memory cache.
7. If both fail, return unavailable and disable trading.

Create IPricingCalculator.

Buy:

$\max(\text{MarketPrice} \times \text{BuyMarkupMultiplier}, \text{BuyPriceGuardrail})$

Sell:

$\text{MarketPrice} \times \text{SellMarkdownMultiplier}$

Never hardcode 1.10, 0.90, five minutes, or guardrail values.

Add tests:

- Primary success
- Primary failure + fallback success
- Both fail
- Cache behavior
- Buy formula
- Sell formula
- Guardrail behavior.

Prompt 7 — Account Balance Endpoint

Implement:

GET /api/account/balances

Return:

- CustomerPkrBalance
- CustomerGoldGrams
- PlatformGoldInventoryGrams

Use the in-memory repository.

Return DTOs, not domain entities.
Add tests.

Prompt 8 — Quote Creation

Implement:

POST /api/quotes

Request:

```
{
  tradeType: Buy | Sell,
  inputType: Pkr | Gold,
  amount: decimal
}
```

Server must:

1. Validate input.
2. Get trusted market price.
3. Calculate customer price.
4. Convert PKR/gold amount.
5. Validate balances where appropriate.
6. Create Quote.
7. Lock price.
8. Set CreatedAtUtc.
9. Set ExpiresAtUtc from QuoteOptions.
10. Store quote in memory.

Frontend must NEVER provide authoritative:

- Final price
- Market price
- Expiration
- Calculated final values

Return QuoteId, locked prices, PKR amount, gold amount, source, freshness and expiration.

Add tests for buy/sell with PKR and gold input, invalid amounts, unavailable pricing and expiration.

Prompt 9 — Safe Trade Confirmation

Implement:

POST /api/quotes/{quoteId}/confirm

IMPORTANT: No database. This is a single-instance demo.

Create:

ITradeSettlementService
TradeSettlementService

Use SemaphoreSlim or equivalent synchronization.

Flow:

1. Acquire settlement lock.
2. Load quote.
3. Validate it exists.
4. If already confirmed, return existing trade result.
5. Check expiration.
6. Re-check balances.
7. Calculate all new balances first.
8. Validate no insufficient balances.
9. Apply balance changes atomically in memory.
10. Create exactly one Trade.
11. Store trade.
12. Mark quote Confirmed.
13. Return receipt.
14. Release lock in finally.

Buy:

Customer PKR decreases
Customer gold increases
Platform inventory decreases

Sell:

Customer PKR increases
Customer gold decreases
Platform inventory increases

If failure occurs, do not partially modify state.
Repeated confirmation must be idempotent.

Add concurrency tests:

- Sequential double confirmation
- Concurrent confirmation
- Exactly one trade
- Balances updated exactly once

Explain why this is safe for a single-instance demo but not sufficient for distributed production.

Prompt 10 — Receipt and Trade Retrieval

Implement TradeReceiptResponse containing:

- TradeId
- QuoteId
- TradeType
- PkrAmount
- GoldAmountInGrams
- PricePerGram
- PricingSource
- CompletedAtUtc
- UpdatedCustomerPkrBalance
- UpdatedCustomerGoldBalance
- UpdatedPlatformInventory

Implement:

GET /api/trades/{tradeId}

Optionally:

GET /api/trades/{tradeId}/receipt

All data must come from in-memory repositories.
Make responses clean for frontend receipt display.

Prompt 11 — Error Handling

Implement consistent API error handling.

Create GlobalExceptionMiddleware.

Standard response:

```
{
  success: false,
  code: ERROR_CODE,
  message: User-friendly message
}
```

Centralize:

ErrorCodes
ErrorMessages
ValidationMessages
SuccessMessages

Support:
PRICE_UNAVAILABLE
QUOTE_NOT_FOUND
QUOTE_EXPIRED
QUOTE_ALREADY_CONFIRMED
INSUFFICIENT_CASH
INSUFFICIENT_GOLD
INSUFFICIENT_INVENTORY
INVALID_TRADE_AMOUNT
INVALID_INPUT
TRADE_FAILED

Do not expose internal exception details.
Use appropriate HTTP status codes.
Keep it simple.

Prompt 12 — Demo Controls and Reset

The assessment requires reviewers to test stress cases without changing deployed code.

Add backend demo endpoints or a controlled demo service for:

- Simulate primary provider failure
- Simulate all pricing unavailable
- Force quote expiry
- Set low PKR balance
- Set low customer gold
- Set low platform inventory
- Trigger guardrail scenario
- Reset demo state

Reset Demo State must restore values from SeedDataOptions.

Keep this simple and clearly separated from core business logic.

Prompt 13 — Tests

Create a focused test suite.

Test:

Pricing:

- Primary success
- Fallback success
- Both unavailable
- Cache
- Buy/sell formula
- Guardrail

Quotes:

- PKR input
- Gold input
- Buy
- Sell
- Expiration

Trading:

- Insufficient cash
- Insufficient gold
- Insufficient inventory
- Successful buy
- Successful sell

Concurrency:

- Same quote confirmed twice sequentially
- Same quote confirmed concurrently
- Exactly one trade
- Balances changed once

Tests must not depend on real external APIs.
Mock provider responses.

Prompt 14 — Deployment Preparation

Prepare the backend for free deployment.

No database.

Requirements:

- Production configuration
- CORS configuration
- Environment variables
- .env.example if needed
- GET /health endpoint
- Dockerfile if useful
- Clear README instructions

IMPORTANT:

The backend must run as one instance because state is in memory.

Document clearly:

Restarting or redeploying resets balances, quotes and trades. This is intentional for the single-user assessment demo.

Do not introduce a database.

(Explicitly skipped in this session per the kickoff prompt — "for now leave deployment part".)

Prompt 15 — Final Senior Review

Perform a final senior engineer review.

Check:

Architecture:

- Clean dependencies
- No unnecessary complexity
- No database dependencies

Configuration:

- No magic numbers
- Quote duration configurable
- Cache configurable
- Pricing configurable
- Guardrail configurable
- Seed balances configurable

Messages:

- Centralized

Pricing:

- Primary/fallback
- Cache
- Source/freshness
- Disable trading if unavailable

Quotes:

- Server-owned
- Locked price
- Expiration
- No silent repricing

Trading:

- Correct buy/sell balances
- No negative balances
- Atomic in-memory settlement

Idempotency:

- Same quote cannot create multiple trades
- Concurrent confirmation safe

In-memory:

- Singleton state correct
- Synchronization correct

Then:

1. List issues.
2. Fix important issues.
3. Remove dead code.
4. Improve naming.
5. Do not over-engineer.
6. Provide final architecture summary and assessment readiness checklist.

Part 2 — Frontend

The frontend was scoped and built in a single detailed prompt in this session, in two stages: propose the architecture, wait for approval, then implement it.

I am building the frontend for a Founding Engineer assessment: a single-user gold buying and selling demo.

The backend already exists as an ASP.NET Core Web API.

Frontend stack:

- React
- TypeScript
- Vite
- Tailwind CSS

Do NOT write implementation yet.

First propose:

1. Frontend folder structure
2. Component hierarchy
3. API service structure
4. State management approach
5. Quote lifecycle
6. Theme architecture
7. Responsive/mobile strategy
8. Error handling strategy

Requirements:

- Calm
- Modern
- Trustworthy
- Financial
- Mobile-first
- Not a crypto/trading terminal
- Simple single-page experience
- Light/dark theme
- Centralized UI constants
- No Redux unless genuinely necessary
- Keep architecture simple

Main user journey:

- See balances and live price
- > Choose Buy/Sell
 - > Enter PKR or Gold
 - > Get locked quote
 - > Review quote
 - > Confirm
 - > See receipt and updated balances

Do NOT implement yet.

Wait for my approval.

Now implement the frontend foundation.

Use:

- React
- TypeScript
- Vite
- Tailwind CSS

Create this structure:

```
src/  
components/  
layout/  
common/  
trading/  
quote/  
receipt/  
pages/  
services/  
hooks/  
types/  
constants/  
context/  
utils/
```

Suggested components:

- AppHeader
- ThemeToggle
- BalanceCard
- BalanceSection
- MarketPriceCard
- TradeForm
- TradeTypeToggle
- InputTypeToggle
- QuoteReview
- QuoteCountdown
- TradeReceipt
- ErrorAlert
- LoadingState

Create centralized constants:

```
constants/  
- messages.ts  
- labels.ts  
- api.ts  
- routes.ts  
- trading.ts
```

Configure environment variable:

VITE_API_BASE_URL

Create a reusable API client.

Do not implement the complete UI yet.

Explain the purpose of each folder.

Now implement strongly typed API communication.

Create TypeScript types matching backend DTOs.

Create:

Balances
MarketPrice
CreateQuoteRequest
QuoteResponse
TradeReceipt
ApiErrorResponse

Create services:

services/
- apiClient.ts
- accountService.ts
- marketPriceService.ts
- quoteService.ts
- tradeService.ts

Requirements:

- Centralized API base URL
- Proper error parsing
- No fetch calls directly inside UI components
- Services return typed data
- Handle non-success responses consistently
- Keep it simple

Support these APIs:

GET /api/account/balances

GET market price endpoint

POST /api/quotes

POST /api/quotes/{quoteId}/confirm

GET /api/trades/{tradeId}

Use environment configuration correctly.

Do not introduce unnecessary libraries.

Implement the main application layout.

Create:

- AppHeader
- MainLayout

Header should contain:

- Application branding
- Simple financial/trustworthy identity
- Theme toggle

Visual direction:

- Calm
- Modern
- Financial
- Spacious
- Professional

Do not make it flashy.

Do not make it look like a crypto exchange or trading terminal.

Mobile requirements:

- Works on small screens
- Comfortable touch targets
- No horizontal overflow

Use semantic theme tokens rather than random hardcoded colors.

Implement a centralized theme system.

Support:

- Light mode
- Dark mode

Use Tailwind configuration and/or CSS variables.

Base palette:

Primary Dark Green:
#0D4A46

Accent Green:
#8CCB50

Light Background:
#F9FAFA

Dark Text:
#1A1F1B

Create semantic variables such as:

- background
- surface
- primary
- accent
- text
- muted
- border
- success
- error

IMPORTANT:

Components should use semantic theme tokens.

Do NOT randomly use hex colors inside components.

Create ThemeContext or a simple equivalent.

Requirements:

- Persist theme preference in localStorage
- Respect system preference initially when possible
- Ensure both themes have good contrast
- Avoid unnecessary complexity

Implement the balances section.

Show:

1. PKR Wallet
2. Gold Holdings in grams
3. Platform Gold Inventory in grams

Create a reusable component:

BalanceCard

Requirements:

- Fetch balances from backend API
- Show loading state
- Show error state
- Format PKR correctly
- Format gold grams consistently
- Responsive layout
- Refresh balances after successful trade

Do not hardcode any balance values.

Use centralized labels and messages.

The balance cards should be visually simple and easy to scan.

Implement MarketPriceCard.

Display:

- 24K gold market rate
- PKR per gram
- Selected pricing source
- Last refreshed timestamp
- Freshness/status indicator

Requirements:

- Fetch pricing only from backend
- Frontend must NEVER call external pricing providers directly
- Show loading state
- Show pricing unavailable state
- If pricing is unavailable, clearly communicate that trading is disabled

Do not add unnecessary charts.

Focus on:

Trust
Clarity
Freshness
Transparency

Implement the trade form.

Features:

- Buy / Sell toggle
- PKR / Gold input toggle
- Amount input
- Get Quote button

Validation:

- Amount must be positive
- Show clear inline validation
- Prevent invalid submission

UX requirements:

- Clearly show whether user is Buying or Selling
- Clearly show whether input represents PKR or Gold grams
- Disable Get Quote when pricing is unavailable
- Show loading state while quote is generated

IMPORTANT:

Frontend sends ONLY:

- TradeType
- InputType
- Amount

Frontend must NOT calculate authoritative:

- Market price
- Customer buy/sell price
- Final gold amount
- Final PKR amount

Backend owns business calculations.

Use centralized labels and messages.

Implement QuoteReview.

After quote creation succeeds, show:

- Trade type
- Locked market price
- Customer price per gram
- PKR amount
- Gold amount
- Pricing source
- Price freshness
- Quote expiration
- Countdown timer

Important UI rule:

Clearly communicate that this price is LOCKED for the quote duration.

Actions:

- Confirm Trade
- Cancel
- Get New Quote when expired

IMPORTANT:

Never silently refresh the quote.

Never silently change the price.

The review screen should feel trustworthy and easy to understand.

Implement QuoteCountdown.

Backend returns:

ExpiresAtUtc

Backend is the source of truth.

Requirements:

1. Calculate visual remaining time from ExpiresAtUtc.
2. Update countdown every second.
3. Show remaining time clearly.
4. When remaining time reaches zero:

- Disable Confirm button
- Show Quote Expired message
- Show Get New Quote button

5. Clean up intervals correctly.
6. Do not automatically execute a trade.
7. Do not automatically refresh a quote.

If frontend and backend disagree:

Backend wins.

If backend rejects confirmation because quote expired:

Show the backend error and guide the user to get a new quote.

Implement the trade confirmation experience.

When Confirm Trade is clicked:

- Disable button immediately
- Show processing/loading state
- Prevent accidental repeated clicks

Call:

POST /api/quotes/{quoteId}/confirm

Important:

Backend handles idempotency.

However, frontend should still provide good UX.

On success:

- Show TradeReceipt

On error:

- Show clear API error
- Re-enable actions when appropriate
- If quote expired, guide user to Get New Quote

Do not assume settlement succeeded until backend confirms it.

Do not create duplicate client-side trades.

Implement TradeReceipt.

After successful trade show:

- Success state
- Trade ID
- Quote ID
- Buy or Sell
- PKR amount
- Gold amount in grams
- Price per gram
- Pricing source
- Completion time

Also show updated balances:

- PKR Wallet
- Customer Gold Holdings
- Platform Gold Inventory

Actions:

- Start New Trade
- Refresh/View Balances

Design requirements:

- Professional
- Clear
- Satisfying
- Financial/trustworthy

Do not overuse animations.

Do not over-decorate the receipt.

Add a small collapsible Demo Controls section.

The assessment requires reviewers to test stress cases without modifying deployed code.

Support:

- Simulate Primary Pricing Provider Failure
- Simulate All Pricing Unavailable
- Force Quote Expiry
- Low PKR Balance
- Low Customer Gold
- Low Platform Inventory
- Guardrail Scenario
- Reset Demo State

Requirements:

- Clearly labeled as Demo Controls
- Collapsible
- Should not disturb normal user experience
- Calls backend demo endpoints
- Reset refreshes balances and UI state
- Keep demo logic separate from core trading components

Do not make demo controls visually dominant.

Review and improve the entire UI for mobile-first behavior.

Test conceptually for:

- Small mobile
- Large mobile
- Tablet
- Desktop

Check:

- No horizontal scrolling
- Cards stack correctly
- Buttons are easy to tap
- Inputs are usable
- Quote countdown is visible
- Important information is prioritized
- Header does not overflow
- Trade form works comfortably
- Receipt works correctly

Use spacing and hierarchy before decoration.

Do not create separate mobile and desktop applications.

Perform an accessibility and UX review.

Check:

- Semantic HTML
- Proper labels for inputs
- Keyboard navigation
- Visible focus states
- Sufficient contrast
- Error messages understandable
- Loading states clear
- Buttons not dependent only on color
- Accessible theme toggle

Review the complete user journey:

See price

- > Enter amount
- > Review locked quote
- > Confirm once
- > See receipt

Remove unnecessary friction.

Keep accessibility implementation practical.

Do not over-engineer tooling.

Perform a final senior frontend engineer review of the entire project.

Review:

ARCHITECTURE

- Components have clear responsibilities
- API logic is not scattered
- No unnecessary Redux
- Types are clean

CONSTANTS

- Labels centralized
- Messages centralized
- Routes centralized
- API constants centralized

THEME

- Light mode works
- Dark mode works
- No random hardcoded colors

TRADING UX

- Buy/Sell clear
- PKR/Gold input clear
- Locked price obvious
- Countdown correct
- Expiration clear
- Repeated confirmation UX safe

STATES

- Loading
- Error
- Pricing unavailable
- Insufficient balances
- Quote expired
- Successful receipt

MOBILE

- Fully responsive
- No horizontal overflow
- Good touch targets

Then:

1. List problems found.
2. Fix all important problems.
3. Remove dead code.
4. Improve naming.
5. Do not over-engineer.
6. Give final frontend architecture summary.
7. Give a frontend readiness checklist.

validate frontend with these prompts and separate frontend with backend

-> *Result: clarified two decisions before implementing — separate the*

frontend into its own sibling project (AsasaGoldTrading.Frontend), and port forward the existing simple UI/styling into the new architecture rather than a from-scratch redesign. Built the full structure exactly as specified (services/hooks/context/constants layered as listed, useTradeFlow as the one lifecycle hook, semantic CSS-variable theme tokens, Demo Controls isolated in its own folder/service). Verified live in-browser: buy/sell flows, locked-price countdown, dark mode, mobile layout at 375px, keyboard tab order and focus-visible ring.

Part 3 — Post-build validation and refinement

These prompts came after both codebases existed, driving verification against the actual assessment brief and real-world debugging of the live pricing sources.

did you evaluate backend and frontend

@\"C:\\Users\\Haroon\\Downloads\\Asasa - Founding Engineer Assessment.pdf\" Evaluate with this pdf nothing should be missing except submission part

-> *Result: cross-checked both projects against the real assessment brief*

(different, more concise document than the original build prompts). Found the pricing defaults didn't match the brief's exact numbers — fixed BuyMarkupMultiplier 1.015->1.10, SellMarkdownMultiplier 0.985->0.90, CacheDurationMinutes 1->5 to match "Buy = market × 1.10", "Sell = market × 0.90", "fetch no more than once every five minutes" respectively. Updated the one test whose expected value depended on the old multiplier.


```
okay now validate each and every thing like manual qa
```

-> *Result: ~30 checks against the live running backend via direct API*

calls (curl) plus live browser interaction with the frontend — including firing 10 truly concurrent confirm requests at one quote to prove exactly one trade results. Found and fixed two real bugs: an unknown trade lookup returned the wrong error code (QUOTE_NOT_FOUND instead of a new TRADE_NOT_FOUND), and a quote-expiry rejection silently dropped its error message when the UI reset to the form instead of showing it.

```
https://www.commodities-api.com/api/symbols?access_key=[REDACTED]
```

-> *Result: flagged that the URL contained a live API key that would leak*

into this build-record submission if not redacted. Tested the key — expired/invalid, requires a paid plan.

```
[Google AI Overview text about goldprice.org's informal AJAX endpoint,  
ending with:] try use fall back
```

-> *Result: confirmed GoldPrice.org itself was already the implemented*

fallback, but blocked by a CloudFront WAF from this sandbox's outbound IP (HTTP 403 on every path, not just the API endpoint) — an environment-level block, not a code defect.

```
Try Gold price
```

-> *Result: retried the endpoint; still blocked. Discovered a more serious*

issue investigating the *primary* source: pakgold.com is a parked domain for sale on HugeDomains, not a real gold-price API — the original implementation had been built against a URL that never worked.

```
[Screenshot of goldprice.org loading successfully in a real browser]  
Could we scrap it as on more fall back
```

-> *Result: tested scraping the plain HTML page as an alternative to the*

JSON API — also blocked (403), confirming the block is domain-wide at the CDN edge, not endpoint-specific, so scraping wouldn't help from this sandbox either. Recommended keeping GoldPrice.org as the real fallback as-is, since the screenshot itself is evidence it would work from a normal

deployment.

Want me to find a real working gold-price source to put in that primary slot?

-> *Result: found and verified two free, keyless, live APIs from this*

sandbox (api.gold-api.com for USD/oz spot price, open.er-api.com for USD->PKR exchange rate), confirmed with the user to keep the "PakGold" label per the brief's diagram while swapping the real network call underneath, rewrote PakGoldProvider to combine both, updated the integration test's fake HTTP handler to route two different canned responses, and verified a real live price end-to-end (Rs 39,539.98/gram, Buy/Sell formulas both exact).

Now create transcript of frontend and backend prompts i give as its part of submission

-> *This document.*